ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES · GÉNIE INFORMATIQUE S8

SecureBox

Conception et réalisation d'un système de surveillance sécurisée d'un répertoire sous Linux

---

Soutenance d'avancement · Année universitaire 2025/2026

Plan de la présentation

- Contexte et problématique
- Objectifs du projet
- Architecture générale choisie
- Structure des fichiers livrés
- Présentation détaillée des modules
- Démonstration en environnement réel
- Difficultés rencontrées et solutions
- Conclusion et perspectives

Contexte du projet

Pourquoi surveiller un répertoire ?

Sous Linux, de nombreux répertoires contiennent des fichiers sensibles : documents administratifs, données de configuration, journaux applicatifs, codes sources. Toute modification non autorisée, suppression accidentelle ou apparition d'un fichier suspect peut compromettre l'intégrité du système ou des données.

Il est donc utile de disposer d'un outil capable de surveiller un répertoire en temps réel, d'enregistrer les événements détectés et de signaler les comportements anormaux. Le projet s'inscrit dans le domaine de la programmation système et met en œuvre les notions fondamentales du cours : gestion des fichiers, threads, signaux, communication inter-processus, et supervision d'événements.

Problématique

Question centrale

> *« Comment concevoir une application système sous Linux capable de surveiller un répertoire en temps réel, d'enregistrer les actions effectuées sur les fichiers et sous-répertoires, et d'identifier certaines activités suspectes — tout en garantissant un fonctionnement fiable, structuré et sécurisé ? »*

Cette problématique guide tous les choix d'architecture et d'implémentation que nous présentons dans les diapositives qui suivent.

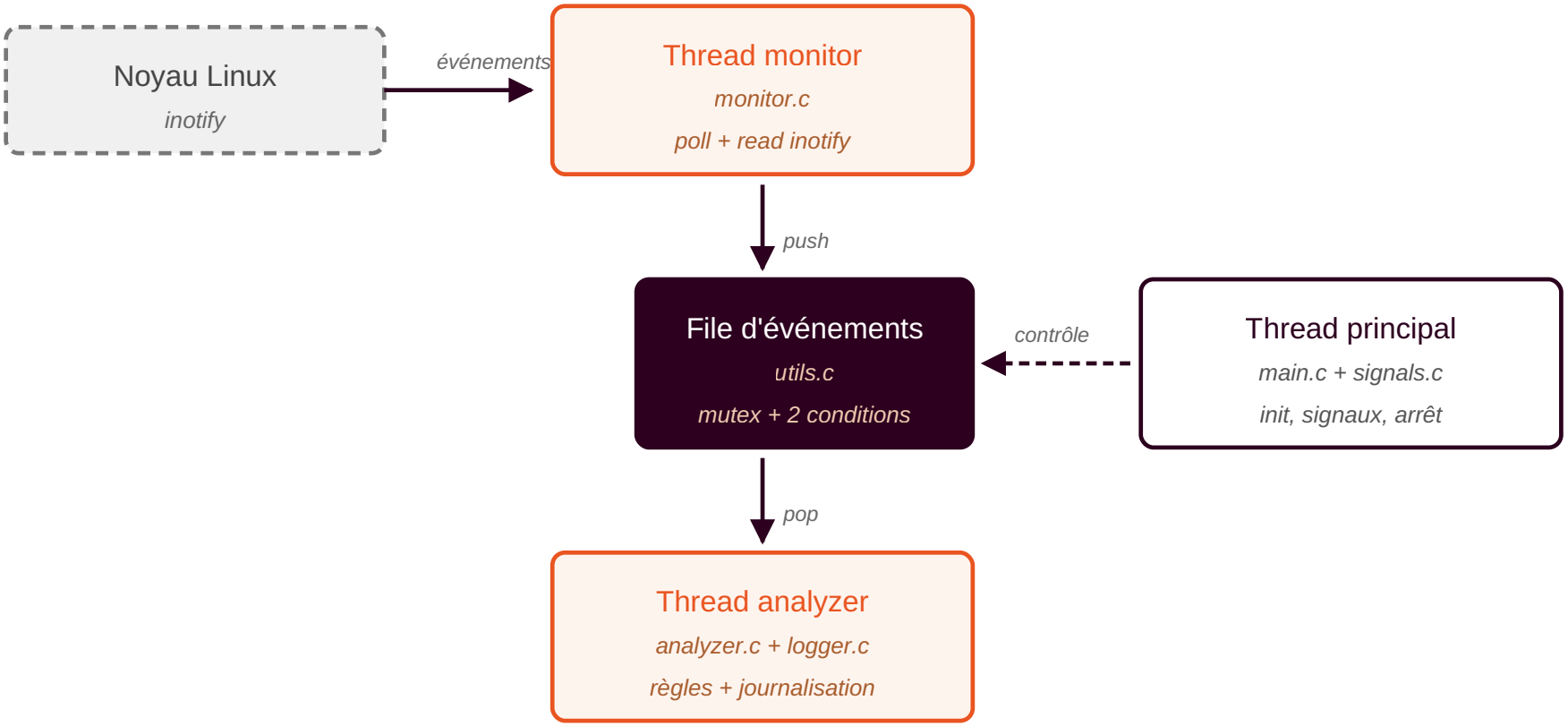
Objectifs

Six fonctionnalités principales

- **Surveillance temps réel** d'un répertoire fourni en argument du programme.
- **Détection des événements** du système de fichiers : création, suppression, modification, renommage, sur fichiers et sous-répertoires.
- **Journalisation horodatée** de chaque événement dans un fichier log structuré.
- **Détection d'anomalies** et émission d'alertes pour les comportements suspects.
- **Statistiques d'exécution** consultables à la demande et à l'arrêt.
- **Arrêt propre** via signaux système avec libération complète des ressources.

Architecture générale

Multi-threads avec file d'événements partagée



Structure du projet

14 fichiers, 7 modules, 1340 lignes

```
securebox/
├── main.c                # point d'entrée, orchestration
├── monitor.c  monitor.h    # surveillance inotify
├── analyzer.c analyzer.h   # règles de détection d'anomalies
├── logger.c   logger.h     # journalisation thread-safe
├── stats.c    stats.h      # compteurs protégés par mutex
├── signals.c  signals.h    # handlers SIGINT/SIGTERM/SIGUSR1
├── utils.c    utils.h      # file thread-safe + helpers
├── config.h                # constantes et seuils
├── Makefile                # compilation, debug, valgrind
├── README.md               # documentation utilisateur
└── logs/
    └── securebox.log       # créé au premier lancement
```

`main.c`

Le chef d'orchestre — 176 lignes

Rôle

Point d'entrée du programme. Quand l'utilisateur tape *./securebox /chemin*, c'est ce fichier qui s'exécute.

Cinq phases dans main()

- **Validation** de l'argument et du chemin (existence, type, droits).
- **Initialisation en cascade** de tous les modules dans l'ordre des dépendances.
- **Lancement des deux threads** de travail (monitor et analyzer).
- **Boucle principale** qui consomme les demandes SIGUSR1 de statistiques.
- **Arrêt propre** : fermeture de la file, join, libération inverse, résumé final.

FONCTIONS CLÉS

- main()
- analyzer_thread()
- usage()

CONCEPTS

- pthread_create
- pthread_join
- static BSS allocation
- cleanup en cascade

`monitor.c` `monitor.h`

L'écouteur inotify — 187 + 31 lignes

Rôle

Seul fichier qui interagit avec le système de fichiers. Demande au noyau Linux de surveiller le répertoire cible, puis traduit les notifications en événements internes.

Comment ça marche

- Ouvre un descripteur **inotify** et y attache un *watch* sur le répertoire à surveiller.
- Tourne en boucle avec **poll() + read()** ; le timeout de 500 ms permet de réagir aux signaux d'arrêt.
- Reconstitue les **renommages internes** en appariant les couples IN_MOVED_FROM / IN_MOVED_TO via leur cookie.
- Pousse chaque événement reconnu dans la file partagée pour le thread d'analyse.

FONCTIONS CLÉS

- monitor_init()
- monitor_run()
- monitor_close()
- push_event()

CONCEPTS

- inotify_init1
- inotify_add_watch
- poll avec timeout
- appariement par cookie

`signals.c` `signals.h`

Le gestionnaire de signaux — 81 + 31 lignes

Rôle

Intercepte les signaux que Linux envoie au processus : Ctrl+C, kill, et SIGUSR1. Les transforme en demandes civilisées que la boucle principale peut consulter à son rythme.

Trois signaux gérés

- **SIGINT** (Ctrl+C) et **SIGTERM** (kill) : demande d'arrêt propre, positionne un drapeau atomique.
- **SIGUSR1** : demande l'affichage immédiat des statistiques sans interrompre la surveillance.
- **SIGPIPE** : ignoré, pour ne pas être tué quand un terminal se ferme.

Principe clé

Les handlers sont minimalistes : ils n'utilisent que des opérations *async-signal-safe*. Le vrai travail (affichage, libération) est fait par le thread principal au prochain réveil.

FONCTIONS CLÉS

- signals_install()
- on_stop()
- on_stats()
- signals_stop_requested()

CONCEPTS

- sigaction
- volatile sig_atomic_t
- async-signal-safety

`utils.c` `utils.h`

La boîte à outils partagée — 170 + 76 lignes

Rôle

Contient deux choses : **la file circulaire thread-safe** (le cœur de l'IPC entre les threads) et plusieurs **utilitaires partagés** (horodatage, vérification de chemin, détection de fichier caché et d'extension interdite).

La file thread-safe

- Tableau circulaire borné (capacité 1024 événements).
- Protégée par un **mutex** et **deux variables de condition** (not_empty, not_full).
- Implémente le pattern classique *producteur / consommateur* avec déblocage propre à l'arrêt via un drapeau closed.

API DE LA FILE

- queue_init / destroy
- queue_push (producteur)
- queue_pop (consommateur)
- queue_close (arrêt)

HELPERS

- utils_format_timestamp
- utils_check_directory
- utils_is_hidden
- utils_has_forbidden_extension

`config.h`

Le tableau de bord — 62 lignes

Rôle

Centralise toutes les **constantes** du projet en un seul endroit. Modifier un seuil ou ajouter une extension interdite ne demande aucune autre modification dans le code.

Ce qu'on y trouve

- Chemins par défaut du répertoire et du fichier de log.
- Tailles maximales des chemins, messages et capacité de la file d'événements.
- Seuils de détection : nombre de suppressions et modifications dans une fenêtre temporelle.
- Liste noire des extensions interdites (.exe, .bat, .sh, .cmd, .dll, .scr).
- Énumérations des types d'événements et des niveaux de gravité.

CONSTANTES CLÉS

- DELETE_THRESHOLD
- DELETE_WINDOW_SEC
- MODIFY_THRESHOLD
- MODIFY_WINDOW_SEC
- EVENT_QUEUE_CAPACITY
- FORBIDDEN_EXTENSIONS

`analyzer.c` `analyzer.h`

Le moteur de règles — 201 + 32 lignes

Rôle

Reçoit chaque événement, lui applique les **cinq règles de détection**, et retourne un niveau de gravité (INFO ou ALERTE) accompagné d'un message descriptif quand une règle se déclenche.

Les cinq règles

- **Rafale de suppressions** : plus de 5 en 10 secondes (fenêtre glissante).
- **Modifications rapides** sur un même fichier : plus de 3 en 5 secondes.
- **Fichier caché** : nom commençant par un point.
- **Extension interdite** : appartenant à la liste noire.
- **Binaire exécutable** : apparition d'un fichier avec un bit d'exécution.

FONCTIONS CLÉS

- analyzer_init / destroy
- analyzer_check()
- record_and_count_deletions()
- find_or_alloc_track()

CONCEPTS

- fenêtre glissante
- table de suivi par fichier
- lstat + masque exec

`logger.c` `logger.h`

La journalisation fiable — 138 + 28 lignes

Rôle

Écrit chaque événement dans le fichier *logs/securebox.log* au format normalisé. Les écritures sont **thread-safe** et résistent à un arrêt brutal grâce à un appel *fsync* après chaque ligne.

Garanties offertes

- Création automatique du répertoire *logs/* et du fichier si nécessaire.
- Droits restrictifs **0640** pour limiter l'accès au propriétaire.
- Écritures atomiques au niveau du noyau via le drapeau **O_APPEND**.
- Aucun message perdu en cas d'arrêt brutal : *fflush* + *fsync* systématiques.
- Double écho : *stdout* pour INFO, *stderr* pour ALERTE.

FONCTIONS CLÉS

- logger_init()
- logger_log_event()
- logger_log_message()
- logger_close()

FORMAT DU LOG

- [date heure]
- [INFO | ALERTE]
- TYPE_EVENEMENT
- chemin complet

`stats.c` `stats.h`

Les compteurs d'activité — 87 + 40 lignes

Rôle

Maintient sept compteurs d'activité **protégés par un mutex**. Permet d'afficher un résumé à la demande (signal SIGUSR1) ou à la fin de l'exécution.

Compteurs maintenus

- Nombre total d'événements détectés depuis le lancement.
- Nombre de créations (fichiers et sous-répertoires).
- Nombre de suppressions.
- Nombre de modifications.
- Nombre de renommages et déplacements internes.
- Nombre d'alertes déclenchées.
- Durée de fonctionnement totale.

FONCTIONS CLÉS

- stats_init / destroy
- stats_record_event()
- stats_record_alert()
- stats_print()

CONCEPTS

- pthread_mutex
- time(NULL) − start_time
- switch sur event_type_t

Démonstration · Lancement

Compilation et démarrage du programme

dani@ubuntu : ~/securebox

```
dani@ubuntu:~/securebox$ make
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c main.c -o main.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c monitor.c -o
monitor.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c logger.c -o
logger.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c analyzer.c -o
analyzer.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c stats.c -o stats.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c signals.c -o
signals.o
gcc -std=c11 -Wall -Wextra -Wpedantic -Wshadow -D_POSIX_C_SOURCE=200809L -c utils.c -o utils.o
gcc  -o securebox main.o monitor.o logger.o analyzer.o stats.o signals.o utils.o -lpthread

dani@ubuntu:~/securebox$ ./securebox /home/dani/test_watched
SecureBox surveille '/home/dani/test_watched'. Ctrl+C pour arreter, SIGUSR1 pour stats.
[2026-05-20 14:32:08] [INFO] SecureBox demarre - cible='/home/dani/test_watched' pid=4521
[2026-05-20 14:32:08] [INFO] Surveillance demarree sur /home/dani/test_watched (pid=4521)
```

Démonstration · Événements normaux

Création, modification et suppression de fichiers

dani@ubuntu : ~/securebox

```
[2026-05-20 14:32:08] [INFO] Surveillance demarree sur /home/dani/test_watched (pid=4521)

# Depuis un second terminal : touch test_watched/data.txt
[2026-05-20 14:32:15] [INFO] CREATE_FILE : /home/dani/test_watched/data.txt

# echo "hello" >> test_watched/data.txt
[2026-05-20 14:32:23] [INFO] MODIFY_FILE : /home/dani/test_watched/data.txt
[2026-05-20 14:32:23] [INFO] MODIFY_FILE : /home/dani/test_watched/data.txt

# mkdir test_watched/sous_dossier
[2026-05-20 14:32:35] [INFO] CREATE_DIR : /home/dani/test_watched/sous_dossier

# mv test_watched/data.txt test_watched/renomme.txt
[2026-05-20 14:32:42] [INFO] RENAME_FILE : /home/dani/test_watched/data.txt -> /home/dani/
test_watched/renomme.txt

# rm test_watched/renomme.txt
[2026-05-20 14:32:45] [INFO] DELETE_FILE : /home/dani/test_watched/renomme.txt
```

Démonstration · Détection d'anomalies

Trois règles déclenchées : fichier caché, extension interdite, rafale

dani@ubuntu : ~/securebox

```
# touch test_watched/.secret
[2026-05-20 14:33:12] [ALERTE] CREATE_FILE : /home/dani/test_watched/.secret
[2026-05-20 14:33:12] [ALERTE] Creation d'un fichier cache : /home/dani/test_watched/.secret

# touch test_watched/malware.exe
[2026-05-20 14:33:30] [ALERTE] CREATE_FILE : /home/dani/test_watched/malware.exe
[2026-05-20 14:33:30] [ALERTE] Extension interdite detectee : /home/dani/test_watched/
malware.exe

# for i in {1..8}; do touch test_watched/f$i; done
# for i in {1..8}; do rm test_watched/f$i; done
[2026-05-20 14:35:21] [INFO] DELETE_FILE : /home/dani/test_watched/f1
[2026-05-20 14:35:21] [INFO] DELETE_FILE : /home/dani/test_watched/f2
[2026-05-20 14:35:21] [INFO] DELETE_FILE : /home/dani/test_watched/f3
[2026-05-20 14:35:21] [INFO] DELETE_FILE : /home/dani/test_watched/f4
[2026-05-20 14:35:22] [INFO] DELETE_FILE : /home/dani/test_watched/f5
[2026-05-20 14:35:22] [ALERTE] DELETE_FILE : /home/dani/test_watched/f6
[2026-05-20 14:35:22] [ALERTE] Activite suspecte detectee : 6 suppressions en 10 secondes
```

Démonstration · Statistiques à la demande

Réception du signal SIGUSR1 depuis un autre terminal

dani@ubuntu : ~/securebox

```
[2026-05-20 14:35:22] [ALERTE] Activite suspecte detectee : 6 suppressions en 10 secondes

# Depuis un autre terminal :  kill -USR1 4521

========== STATISTIQUES SECUREBOX ==========
 Evenements totaux        : 32
 Creations                : 11
 Suppressions             : 9
 Modifications            : 7
 Renommages / deplacements: 1
 Alertes generees         : 5
 Duree de fonctionnement  : 0h 03m 14s
============================================
[2026-05-20 14:35:33] [INFO] Statistiques affichees via SIGUSR1.
```

Démonstration · Arrêt propre

Interruption par Ctrl+C et libération ordonnée des ressources

dani@ubuntu : ~/securebox

```
[2026-05-20 14:35:33] [INFO] Statistiques affichees via SIGUSR1.
^C
[2026-05-20 14:35:50] [INFO] Arret demande, fermeture en cours...
[2026-05-20 14:35:50] [INFO] Thread d'analyse termine.
[2026-05-20 14:35:50] [INFO] Thread de surveillance termine.

========== STATISTIQUES SECUREBOX ==========
 Evenements totaux        : 32
 Creations                : 11
 Suppressions             : 9
 Modifications            : 7
 Renommages / deplacements: 1
 Alertes generees         : 5
 Duree de fonctionnement  : 0h 03m 42s
============================================
[2026-05-20 14:35:50] [INFO] SecureBox arrete - evenements=32 alertes=5

dani@ubuntu:~/securebox$
```

Difficultés rencontrées

Cinq problèmes et leurs solutions

Dépassement de pile au démarrage

La file d'événements (~8 Mo) déclarée en variable locale dans main() saturait la pile par défaut de Linux. Le programme se terminait sur SIGSEGV avant même l'initialisation. **Solution :** déclaration *static* pour la placer dans le segment BSS.

Appariement des renommages

inotify émet deux événements pour un seul renommage (IN_MOVED_FROM puis IN_MOVED_TO), liés par un cookie numérique. **Solution :** structure pending_move_t locale qui mémorise le premier en attendant le second.

Signaux et threads

Un handler de signal ne peut presque rien faire en sécurité (printf, malloc et mutex sont interdits). **Solution :** handlers minimalistes qui ne font que positionner un volatile sig_atomic_t, le vrai travail étant fait par la boucle principale.

Persistance du journal

Un simple *fflush* ne garantit pas l'écriture sur disque : les données restent en cache noyau et peuvent être perdues en cas d'arrêt brutal. **Solution :** appel *fsync* systématique après chaque ligne du journal.

Réveil propre des threads

Avec deux threads bloquées dans pthread_cond_wait, comment les réveiller proprement à l'arrêt ? **Solution :** un drapeau *closed* sur la file, combiné à un pthread_cond_broadcast émis par queue_close.

Tests sans terminal interactif

Tester le programme dans un environnement automatisé sans terminal humain. **Solution :** lancement en arrière-plan, déclenchement des événements via un second shell, et envoi de signaux via la commande *kill* ciblant le PID.

Conclusion

SecureBox satisfait tous les objectifs du cahier des charges

10/10

Tests passés

Les dix scénarios du cahier des charges ont été exécutés avec succès, sans aucune perte d'événement ni crash.

0

Avertissements

Compilation propre avec -Wall -Wextra -Wpedantic -Wshadow. Aucune fuite mémoire détectée avec Valgrind.

1340

Lignes de code

Réparties sur sept modules indépendants. Code structuré, lisible, et entièrement documenté.

Perspectives d'évolution

Surveillance récursive des sous-répertoires · Quarantaine automatique des fichiers suspects · Export du journal au format CSV · Interface texte avec ncurses · Chiffrement du fichier journal · Filtrage des événements par type ou gravité.